

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ

к лабораторной работе №2

на тему:

«СИММЕТРИЧНАЯ КРИПТОГРАФИЯ. СТБ 34.101.31-2011»

БГУИР 6-05-0612-02 74

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

Проверил ассистент каф. Информатики
ПОЛЯКОВ Кирилл Борисович

Минск 2026

СОДЕРЖАНИЕ

Введение.....	3
1 Выполнение работы	5
Вывод.....	8
Приложение А (обязательное) Исходный код программы	9

ВВЕДЕНИЕ

Настоящий стандарт определяет семейство криптографических алгоритмов, предназначенных для обеспечения конфиденциальности и контроля целостности данных. Обрабатываемыми данными являются двоичные слова.

Криптографические алгоритмы шифрования и контроля целостности делятся на восемь групп:

- алгоритмы шифрования в режиме простой замены;
- алгоритмы шифрования в режиме сцепления блоков;
- алгоритмы шифрования в режиме гаммирования с обратной связью;
- алгоритмы шифрования в режиме счетчика;
- алгоритм выработки имитовставки;
- алгоритмы одновременного шифрования и имитозащиты данных;
- алгоритмы одновременного шифрования и имитозащиты ключа;
- алгоритм хеширования.

Первые четыре группы предназначены для обеспечения конфиденциальности сообщений. Каждая группа включает алгоритм зашифрования и алгоритм расшифрования.

Стороны, располагающие общим ключом, могут организовать конфиденциальный обмен сообщениями путем их зашифрования перед отправкой и расшифрования после получения. В режимах простой замены и сцепления блоков шифруются сообщения, которые содержат хотя бы один блок, а в режимах гаммирования с обратной связью и счетчика – сообщения произвольной длины.

Пятый алгоритм предназначен для контроля целостности сообщений с помощью имитовставок – контрольных слов, которые определяются с использованием ключа.

В данной лабораторной работе рассматриваются два режима шифрования: режим простой замены и режим гаммирования с обратной связью.

Входными данными алгоритма блочного шифрования являются блок данных размером 128 бит и секретный ключ размером 256 бит. Блок данных представляется в виде четырех 32-битных слов $X = X_1 \parallel X_2 \parallel X_3 \parallel X_4$.

Ключ представляется в виде восьми 32-битных слов: $\theta = \theta_1 \parallel \theta_2 \parallel \theta_3 \parallel \theta_4 \parallel \theta_5 \parallel \theta_6 \parallel \theta_7 \parallel \theta_8$.

На основании этих восьми слов формируются тактовые ключи $K_1 = \theta_1$, $K_2 = \theta_2$, $K_3 = \theta_3$, $K_4 = \theta_4$, $K_5 = \theta_5$, $K_6 = \theta_6$, $K_7 = \theta_7$, $K_8 = \theta_8$, $K_9 = \theta_1$, ..., $K_{56} = \theta_8$. В процессе шифрования выполняется восемь тактов преобразования, каждый из которых содержит несколько операций сложения и вычитания по модулю 2^{32} , побитового исключающего ИЛИ, нелинейного преобразования с использованием таблицы подстановок и перестановки промежуточных слов.

Основным преобразованием является функция H , которая выполняет замену каждого 8-битного фрагмента входного слова в соответствии с фиксированной таблицей подстановок. После выполнения подстановок

результат циклически сдвигается влево на заданное количество бит. Полученное преобразование используется в функциях G_5 , G_{13} и G_{21} .

В режиме простой замены каждый блок исходного сообщения независимо преобразуется с использованием блочного алгоритма шифрования. В режиме гаммирования с обратной связью для формирования гаммы используется результат шифрования предыдущего блока. Первый блок обрабатывается с использованием синхропосылки размером 128 бит.

Задание:

1 Изучить теоретические сведения.

2 Реализовать программные средства шифрования и дешифрования текстовых файлов при помощи стандарта шифрования СТБ 34.101.31-2011 в режимах:

- простой замены;
- гаммирования с обратной связью.

1 ВЫПОЛНЕНИЕ РАБОТЫ

Для алгоритма шифрования СТБ 34.101.31-2011 в режиме простой замены сделана блок-схема, представленная на рисунке 1.1.

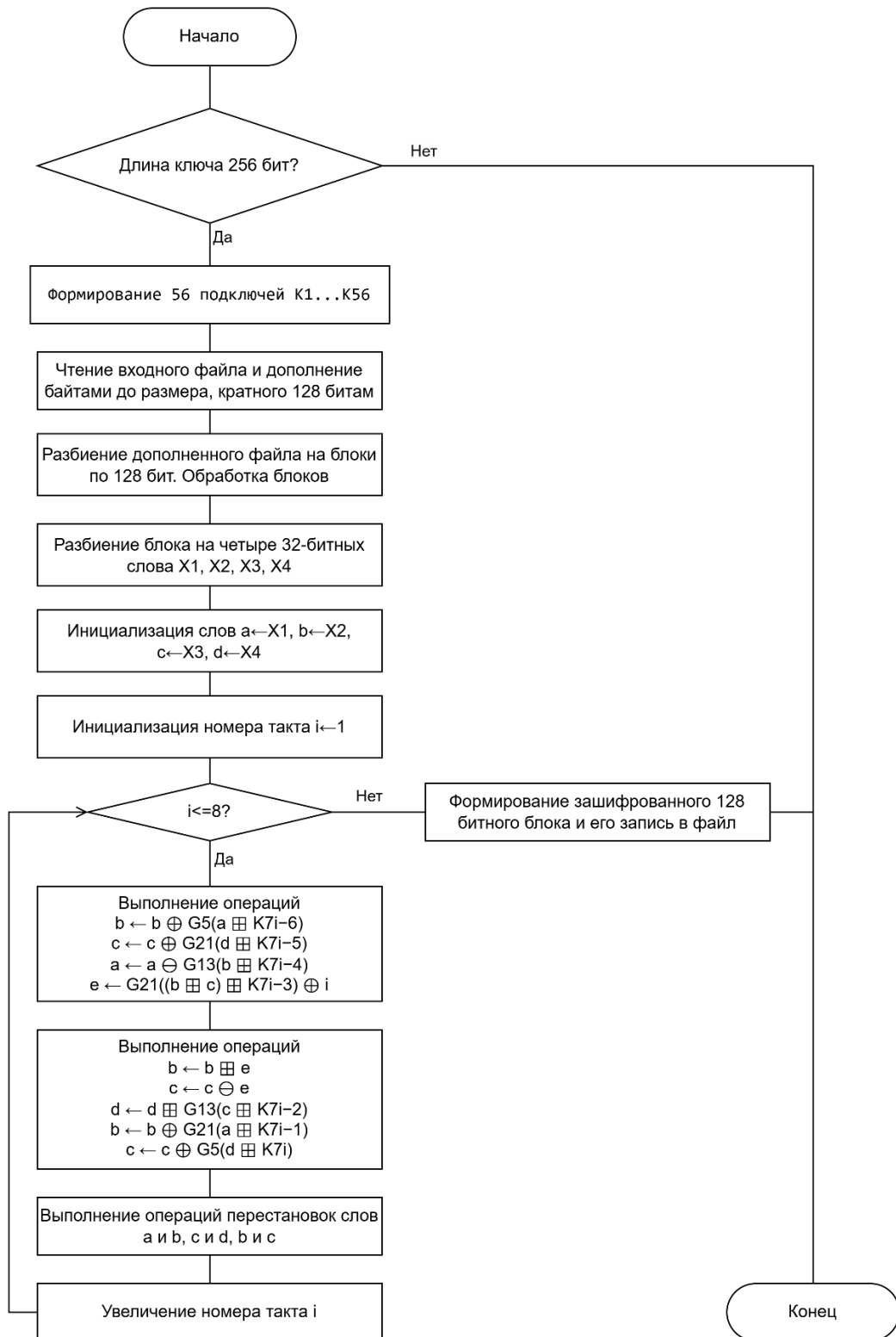
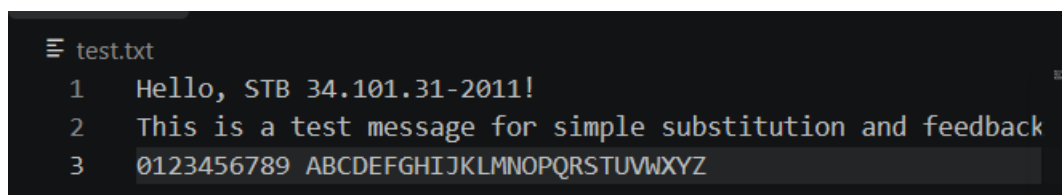


Рисунок 1.1 – Блок-схема алгоритма шифрования СТБ 34.101.31-2011

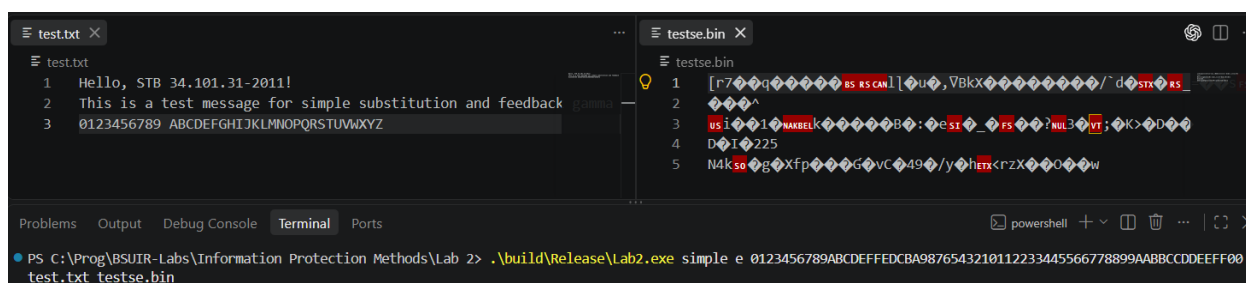
Для проверки работы разработанного алгоритма во всех режимах подается файл test.txt, содержимое которого представлено на рисунке 1.2.



```
test.txt
1 Hello, STB 34.101.31-2011!
2 This is a test message for simple substitution and feedback
3 0123456789 ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

Рисунок 1.2 – Содержимое файла test.txt

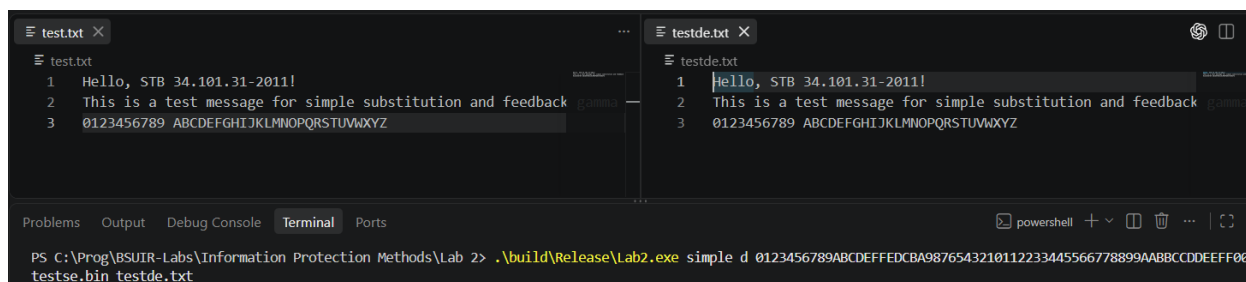
Сперва программа была запущена в режиме шифрования методом простой замены. Результат выполнения работы программы представлен на рисунке 1.3.



The screenshot shows a debugger window with two tabs: test.txt and testse.bin. The test.txt tab shows the original text. The testse.bin tab shows the encrypted output, which is a series of random-looking characters. The terminal at the bottom shows the command: `PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 2> .\build\Release\Lab2.exe simple e 0123456789ABCDEFFEDCBA9876543210112233445566778899AABCCDDEEFF00 test.txt testse.bin`

Рисунок 1.3 – Результат выполнения работы программы в режиме шифрования методом простой замены

Далее программа была запущена в режиме дешифрования методом простой замены. Результат выполнения работы представлен на рисунке 1.4.



The screenshot shows a debugger window with two tabs: test.txt and testde.txt. The test.txt tab shows the original text. The testde.txt tab shows the decrypted output, which is identical to the original text. The terminal at the bottom shows the command: `PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 2> .\build\Release\Lab2.exe simple d 0123456789ABCDEFFEDCBA9876543210112233445566778899AABCCDDEEFF00 testse.bin testde.txt`

Рисунок 1.4 – Результат выполнения работы программы в режиме дешифрования методом простой замены

После программа была запущена в режиме шифрования методом гаммирования с обратной связью. Результат выполнения работы программы представлен на рисунке 1.5.

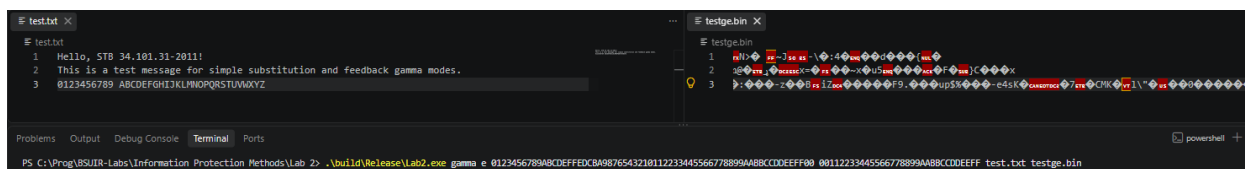


Рисунок 1.5 – Результат выполнения работы программы в режиме шифрования методом гаммирования с обратной связью

Наконец программа была запущена в режиме дешифрования методом гаммирования с обратной связью. Результат выполнения работы представлен на рисунке 1.6.

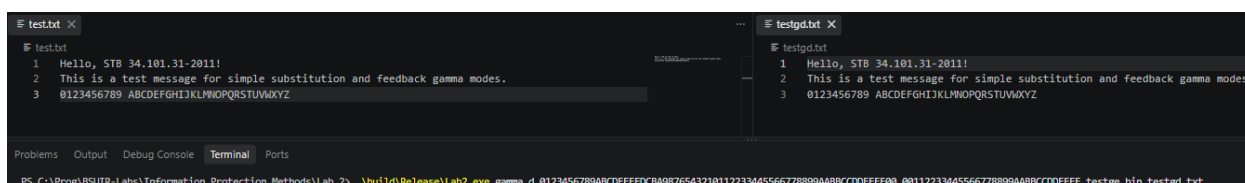


Рисунок 1.6 – Результат выполнения работы программы в режиме дешифрования методом гаммирования с обратной связью

ВЫВОД

В ходе выполнения лабораторной работы был изучен стандарт симметричного блочного шифрования СТБ 34.101.31-2011, предназначенный для обеспечения конфиденциальности и контроля целостности данных. Были рассмотрены основные принципы работы стандарта, структура обрабатываемого 128-битного блока данных, структура 256-битного секретного ключа, порядок формирования тактовых ключей, таблица нелинейной подстановки и основные операции, используемые при выполнении раундовых преобразований.

В рамках работы были изучены алгоритмы шифрования в режиме простой замены и в режиме гаммирования с обратной связью.

В результате выполнения лабораторной работы было разработано программное средство, реализующее шифрование и дешифрование текстовых файлов в режимах простой замены и гаммирования с обратной связью.

Корректность работы программы была проверена на тестовом файле. Для каждого из реализованных режимов выполнялось последовательное шифрование исходного файла и его последующее дешифрование. В результате дешифрования было получено исходное содержимое файла, что подтверждает корректность реализации операций шифрования и дешифрования.

Таким образом, в ходе лабораторной работы были закреплены практические навыки применения симметричных криптографических алгоритмов, работы с блочными шифрами, реализации различных режимов шифрования и программной обработки файлов с использованием стандарта СТБ 34.101.31-2011.

ПРИЛОЖЕНИЕ А

(обязательное)

Исходный код программы

```
#include <stdint.h>
#include <array>

using KEY = std::array<uint32_t, 8>;
using KTABLE = std::array<uint8_t, 256>;
using DBLOCK = std::array<uint32_t, 4>;
using SUBKEYS = std::array<uint32_t, 56>;

#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <algorithm>

#include "STBStructures.h"

constexpr KTABLE ktable={
0xB1, 0x94, 0xBA, 0xC8, 0x0A, 0x08, 0xF5, 0x3B, 0x36, 0x6D, 0x00, 0x8E,
0x58, 0x4A, 0x5D, 0xE4,
0x85, 0x04, 0xFA, 0x9D, 0x1B, 0xB6, 0xC7, 0xAC, 0x25, 0x2E, 0x72, 0xC2,
0x02, 0xFD, 0xCE, 0x0D,
0x5B, 0xE3, 0xD6, 0x12, 0x17, 0xB9, 0x61, 0x81, 0xFE, 0x67, 0x86, 0xAD,
0x71, 0x6B, 0x89, 0x0B,
0x5C, 0xB0, 0xC0, 0xFF, 0x33, 0xC3, 0x56, 0xB8, 0x35, 0xC4, 0x05, 0xAE,
0xD8, 0xE0, 0x7F, 0x99,
0xE1, 0x2B, 0xDC, 0x1A, 0xE2, 0x82, 0x57, 0xEC, 0x70, 0x3F, 0xCC, 0xF0,
0x95, 0xEE, 0x8D, 0xF1,
0xC1, 0xAB, 0x76, 0x38, 0x9F, 0xE6, 0x78, 0xCA, 0xF7, 0xC6, 0xF8, 0x60,
0xD5, 0xBB, 0x9C, 0x4F,
0xF3, 0x3C, 0x65, 0x7B, 0x63, 0x7C, 0x30, 0x6A, 0xDD, 0x4E, 0xA7, 0x79,
0x9E, 0xB2, 0x3D, 0x31,
0x3E, 0x98, 0xB5, 0x6E, 0x27, 0xD3, 0xBC, 0xCF, 0x59, 0x1E, 0x18, 0x1F,
0x4C, 0x5A, 0xB7, 0x93,
0xE9, 0xDE, 0xE7, 0x2C, 0x8F, 0x0C, 0x0F, 0xA6, 0x2D, 0xDB, 0x49, 0xF4,
0x6F, 0x73, 0x96, 0x47,
0x06, 0x07, 0x53, 0x16, 0xED, 0x24, 0x7A, 0x37, 0x39, 0xCB, 0xA3, 0x83,
0x03, 0xA9, 0x8B, 0xF6,
0x92, 0xBD, 0x9B, 0x1C, 0xE5, 0xD1, 0x41, 0x01, 0x54, 0x45, 0xFB, 0xC9,
0x5E, 0x4D, 0x0E, 0xF2,
0x68, 0x20, 0x80, 0xAA, 0x22, 0x7D, 0x64, 0x2F, 0x26, 0x87, 0xF9, 0x34,
0x90, 0x40, 0x55, 0x11,
0xBE, 0x32, 0x97, 0x13, 0x43, 0xFC, 0x9A, 0x48, 0xA0, 0x2A, 0x88, 0x5F,
0x19, 0x4B, 0x09, 0xA1,
0x7E, 0xCD, 0xA4, 0xD0, 0x15, 0x44, 0xAF, 0x8C, 0xA5, 0x84, 0x50, 0xBF,
0x66, 0xD2, 0xE8, 0x8A,
0xA2, 0xD7, 0x46, 0x52, 0x42, 0xA8, 0xDF, 0xB3, 0x69, 0x74, 0xC5, 0x51,
0xEB, 0x23, 0x29, 0x21,
0xD4, 0xEF, 0xD9, 0xB4, 0x3A, 0x62, 0x28, 0x75, 0x91, 0x14, 0x10, 0xEA,
0x77, 0x6C, 0xDA, 0x1D
};

KEY key;
SUBKEYS sub_keys;
std::vector<DBLOCK> file_data;
std::size_t data_size = 0;

void substitute(uint32_t& x)
```

```

{
    uint8_t a = (x >> 24) & 0xFF;
    uint8_t b = (x >> 16) & 0xFF;
    uint8_t c = (x >> 8) & 0xFF;
    uint8_t d = x & 0xFF;

    x = ((uint32_t)ktable[a] << 8*3) |
        ((uint32_t)ktable[b] << 8*2) |
        ((uint32_t)ktable[c] << 8) |
        ((uint32_t)ktable[d]);
}

void rotl(uint32_t& x, const unsigned r)
{
    uint32_t e = x >> (32 - r);
    x = (x << r) | e;
}

void G(uint32_t& x, const unsigned r)
{
    substitute(x);
    rotl(x, r);
}

uint32_t add_mod(uint32_t x, uint32_t y)
{
    return x + y;
}

uint32_t sub_mod(uint32_t x, uint32_t y)
{
    return x - y;
}

void round(DBLOCK& data, unsigned rn)
{
    uint32_t sk = sub_keys[7 * rn - 7];
    uint32_t tmp = add_mod(data[0], sk);
    G(tmp, 5);
    data[1] ^= tmp;

    sk = sub_keys[7 * rn - 6];
    tmp = add_mod(data[3], sk);
    G(tmp, 21);
    data[2] ^= tmp;

    sk = sub_keys[7 * rn - 5];
    tmp = add_mod(data[1], sk);
    G(tmp, 13);
    data[0] = sub_mod(data[0], tmp);

    sk = sub_keys[7 * rn - 4];
    tmp = add_mod(add_mod(data[1], data[2]), sk);
    G(tmp, 21);
    uint32_t e = tmp ^ static_cast<uint32_t>(rn);

    data[1] = add_mod(data[1], e);
    data[2] = sub_mod(data[2], e);

    sk = sub_keys[7 * rn - 3];
    tmp = add_mod(data[2], sk);
    G(tmp, 13);
    data[3] = add_mod(data[3], tmp);
}

```

```

        sk = sub_keys[7 * rn - 2];
        tmp = add_mod(data[0], sk);
        G(tmp, 21);
        data[1] ^= tmp;

        sk = sub_keys[7 * rn - 1];
        tmp = add_mod(data[3], sk);
        G(tmp, 5);
        data[2] ^= tmp;

        std::swap(data[0], data[1]);
        std::swap(data[2], data[3]);
        std::swap(data[1], data[2]);
    }

void encrypt_block(DBLOCK& data)
{
    for (unsigned i = 1; i <= 8; i++)
    {
        round(data, i);
    }

    DBLOCK result = {
        data[1],
        data[3],
        data[0],
        data[2]
    };

    data = result;
}

void decrypt_block(DBLOCK& data)
{
    DBLOCK state = {
        data[2],
        data[0],
        data[3],
        data[1]
    };

    data = state;

    for (unsigned i = 8; i >= 1; i--)
    {
        std::swap(data[1], data[2]);
        std::swap(data[2], data[3]);
        std::swap(data[0], data[1]);

        uint32_t sk = sub_keys[7 * i - 1];
        uint32_t tmp = add_mod(data[3], sk);
        G(tmp, 5);
        data[2] ^= tmp;

        sk = sub_keys[7 * i - 2];
        tmp = add_mod(data[0], sk);
        G(tmp, 21);
        data[1] ^= tmp;

        sk = sub_keys[7 * i - 3];
        tmp = add_mod(data[2], sk);
        G(tmp, 13);
        data[3] = sub_mod(data[3], tmp);
    }
}

```

```

        sk = sub_keys[7 * i - 4];
        tmp = add_mod(add_mod(data[1], data[2]), sk);
        G(tmp, 21);
        uint32_t e = tmp ^ static_cast<uint32_t>(i);

        data[1] = sub_mod(data[1], e);
        data[2] = add_mod(data[2], e);

        sk = sub_keys[7 * i - 5];
        tmp = add_mod(data[1], sk);
        G(tmp, 13);
        data[0] = add_mod(data[0], tmp);

        sk = sub_keys[7 * i - 6];
        tmp = add_mod(data[3], sk);
        G(tmp, 21);
        data[2] ^= tmp;

        sk = sub_keys[7 * i - 7];
        tmp = add_mod(data[0], sk);
        G(tmp, 5);
        data[1] ^= tmp;
    }
}

bool read_file(const std::string& path, bool add_padding, bool
allow_partial)
{
    std::ifstream fin(path, std::ios::binary);

    file_data.clear();
    data_size = 0;

    if (!fin)
        return false;

    std::vector<uint8_t> bytes;
    uint8_t byte;

    while (fin.read(reinterpret_cast<char*>(&byte), sizeof(byte)))
    {
        bytes.push_back(byte);
    }

    if (add_padding)
    {
        std::size_t remainder = bytes.size() % 16;
        std::size_t padding_size = 16 - remainder;

        for (std::size_t i = 0; i < padding_size; i++)
        {
            bytes.push_back(static_cast<uint8_t>(padding_size));
        }
    }
    else
    {
        if (!allow_partial)
        {
            if (bytes.empty())
                return false;

            if (bytes.size() % 16 != 0)
                return false;
        }
    }
}

```

```

    }

    data_size = bytes.size();

    if (bytes.empty())
        return true;

    std::size_t byte_index = 0;

    while (byte_index < bytes.size())
    {
        DBLOCK block = {0, 0, 0, 0};

        for (unsigned word = 0; word < 4; word++)
        {
            for (unsigned byte_in_word = 0; byte_in_word < 4;
byte_in_word++)
            {
                if (byte_index >= bytes.size())
                    break;

                block[word] |=
                    static_cast<uint32_t>(bytes[byte_index])
                    << (8 * byte_in_word);

                byte_index++;
            }
        }

        file_data.push_back(block);
    }

    return true;
}

bool remove_padding()
{
    if (file_data.empty())
        return false;

    std::vector<uint8_t> bytes;

    for (const auto& block : file_data)
    {
        for (const auto& word : block)
        {
            for (unsigned byte = 0; byte < 4; byte++)
            {
                bytes.push_back(
                    static_cast<uint8_t>((word >> (8 * byte)) & 0xFF)
                );
            }
        }
    }

    if (bytes.empty())
        return false;

    uint8_t padding_size = bytes.back();

    if (padding_size == 0 || padding_size > 16)
        return false;

    if (padding_size > bytes.size())

```

```

        return false;

    for (std::size_t i = 0; i < padding_size; i++)
    {
        if (bytes[bytes.size() - 1 - i] != padding_size)
            return false;
    }

    bytes.resize(bytes.size() - padding_size);

    data_size = bytes.size();

    file_data.clear();

    std::size_t byte_index = 0;

    while (byte_index < bytes.size())
    {
        DBLOCK block = {0, 0, 0, 0};

        for (unsigned word = 0; word < 4; word++)
        {
            for (unsigned byte_in_word = 0; byte_in_word < 4;
byte_in_word++)
            {
                if (byte_index >= bytes.size())
                    break;

                block[word] |=
                    static_cast<uint32_t>(bytes[byte_index])
                    << (8 * byte_in_word);

                byte_index++;
            }
        }

        file_data.push_back(block);
    }

    return true;
}

void write_file(const std::string& path, std::size_t size)
{
    std::ofstream fout(path, std::ios::binary);

    if (!fout)
        return;

    std::size_t written = 0;

    for (const auto& block : file_data)
    {
        for (const auto& word : block)
        {
            for (unsigned byte = 0; byte < 4; byte++)
            {
                if (written >= size)
                    return;

                uint8_t value =
                    static_cast<uint8_t>((word >> (8 * byte)) & 0xFF);

                fout.write(

```

```

        reinterpret_cast<const char*>(&value),
        sizeof(value)
    );

    written++;
}
}
}

void encrypt_file()
{
    for (auto& i : file_data)
    {
        encrypt_block(i);
    }
}

void decrypt_file()
{
    for (auto& i : file_data)
    {
        decrypt_block(i);
    }
}

void gamma_file(DBLOCK sync, bool encryption)
{
    DBLOCK feedback = sync;

    for (std::size_t i = 0; i < file_data.size(); i++)
    {
        DBLOCK previous_ciphertext = file_data[i];

        DBLOCK gamma = feedback;
        encrypt_block(gamma);

        std::size_t block_size =
            std::min<std::size_t>(16, data_size - i * 16);

        for (std::size_t byte = 0; byte < block_size; byte++)
        {
            std::size_t word = byte / 4;
            std::size_t offset = byte % 4;

            uint8_t value =
                static_cast<uint8_t>(
                    (file_data[i][word] >> (8 * offset)) & 0xFF
                );

            uint8_t gamma_byte =
                static_cast<uint8_t>(
                    (gamma[word] >> (8 * offset)) & 0xFF
                );

            value ^= gamma_byte;

            file_data[i][word] &=
                ~(static_cast<uint32_t>(0xFF) << (8 * offset));

            file_data[i][word] |=
                static_cast<uint32_t>(value) << (8 * offset);
        }
    }
}

```

```

        if (encryption)
            feedback = file_data[i];
        else
            feedback = previous_ciphertext;
    }
}

bool parse_key(const std::string& key_string, KEY& key)
{
    if (key_string.size() != 64)
        return false;

    for (unsigned i = 0; i < 8; i++)
    {
        uint32_t word = 0;

        for (unsigned j = 0; j < 8; j++)
        {
            char c = key_string[i * 8 + j];

            uint8_t value;

            if (c >= '0' && c <= '9')
                value = c - '0';
            else if (c >= 'A' && c <= 'F')
                value = c - 'A' + 10;
            else if (c >= 'a' && c <= 'f')
                value = c - 'a' + 10;
            else
                return false;

            word = (word << 4) | value;
        }

        key[i] = word;
    }

    return true;
}

bool parse_sync(const std::string& sync_string, DBLOCK& sync)
{
    if (sync_string.size() != 32)
        return false;

    for (unsigned i = 0; i < 4; i++)
    {
        uint32_t word = 0;

        for (unsigned j = 0; j < 8; j++)
        {
            char c = sync_string[i * 8 + j];

            uint8_t value;

            if (c >= '0' && c <= '9')
                value = c - '0';
            else if (c >= 'A' && c <= 'F')
                value = c - 'A' + 10;
            else if (c >= 'a' && c <= 'f')
                value = c - 'a' + 10;
            else
                return false;
        }
    }
}

```

```

        word = (word << 4) | value;
    }

    sync[i] = word;
}

return true;
}

void expand_key()
{
    for (int i = 0; i < 56; i++)
    {
        sub_keys[i] = key[i % 8];
    }
}

int main(int argc, char* argv[])
{
    if (argc < 6 || argc > 7)
    {
        std::cerr << "Usage: " << argv[0]
                    << " <mode> <operation> <key> [sync] <input_file>
<output_file>\n";
        return 1;
    }

    std::string mode = argv[1];
    std::string operation = argv[2];
    std::string key_string = argv[3];

    std::string sync_string;
    std::string input_file;
    std::string output_file;

    if (mode == "simple")
    {
        if (argc != 6)
        {
            std::cerr << "Usage: " << argv[0]
                        << " simple <operation> <key> <input_file>
<output_file>\n";
            return 1;
        }

        input_file = argv[4];
        output_file = argv[5];
    }
    else if (mode == "gamma")
    {
        if (argc != 7)
        {
            std::cerr << "Usage: " << argv[0]
                        << " gamma <operation> <key> <sync> <input_file>
<output_file>\n";
            return 1;
        }

        sync_string = argv[4];
        input_file = argv[5];
        output_file = argv[6];
    }
    else
    {

```

```

        std::cerr << "Error: mode must be 'simple' or 'gamma'\n";
        return 1;
    }

    if (operation != "e" && operation != "d")
    {
        std::cerr << "Error: operation must be 'e' or 'd'\n";
        return 1;
    }

    if (!parse_key(key_string, key))
    {
        std::cerr << "Error: key must contain exactly 64 hexadecimal
characters\n";
        return 1;
    }

    expand_key();

    DBLOCK sync = {0, 0, 0, 0};

    if (mode == "gamma")
    {
        if (!parse_sync(sync_string, sync))
        {
            std::cerr << "Error: sync must contain exactly 32 hexadecimal
characters\n";
            return 1;
        }
    }

    if (mode == "simple")
    {
        if (operation == "e")
        {
            if (!read_file(input_file, true, false))
            {
                std::cerr << "Error: failed to read input file\n";
                return 1;
            }

            if (file_data.empty())
            {
                std::cerr << "Error: failed to read input file\n";
                return 1;
            }

            encrypt_file();
        }
        else
        {
            if (!read_file(input_file, false, false))
            {
                std::cerr << "Error: failed to read input file\n";
                return 1;
            }

            if (file_data.empty())
            {
                std::cerr << "Error: failed to read input file\n";
                return 1;
            }

            decrypt_file();
        }
    }

```

```

        if (!remove_padding())
        {
            std::cerr << "Error: invalid padding or corrupted input
file\n";
            return 1;
        }
    }
else
{
    if (!read_file(input_file, false, true))
    {
        std::cerr << "Error: failed to read input file\n";
        return 1;
    }

    gamma_file(sync, operation == "e");
}

write_file(output_file, data_size);

return 0;
}

```